

P1

Personne 1 — celui/celle qui connaît Flask

Le moteur de l'application · Backend complet

+ expérimenté

Créer la structure du projet Flask

Initialiser le dossier du projet avec les bons sous-dossiers, créer le fichier principal `app.py`, configurer Flask (clé secrète, dossier d'upload, connexion SQLite). C'est la base sur laquelle tout le monde travaille — ça doit être fait en premier.

Fichiers : `app.py` · `config.py`

Créer la base de données SQLite et toutes les tables

Écrire le script Python qui crée les 4 tables : `users` (`id`, `nom`, `email`, `mot_de_passe`, `téléphone`), `annonces` (`id`, `titre`, `description`, `prix`, `surface`, `wilaya`, `type`, `statut`, `user_id`, `date`), `photos` (`id`, `annonce_id`, `chemin_fichier`), `sessions` gérées par Flask. Définir les clés primaires et étrangères.

Fichiers : `models.py` · `init_db.py` · `app.db`

Routes d'authentification (inscription, connexion, déconnexion)

Écrire les 3 routes Flask : GET/POST `/inscription` (créer un compte, hacher le mot de passe avec `werkzeug`), GET/POST `/connexion` (vérifier le mot de passe haché, stocker l'`user_id` en session), GET `/deconnexion` (vider la session). Protéger les pages qui nécessitent d'être connecté avec un décorateur `@login_required`.

Fichiers : `app.py` · `auth.py`

Routes pour les annonces (publier, modifier, supprimer)

Route POST `/publier` : récupérer les données du formulaire, valider les champs obligatoires, insérer dans la table `annonces` avec statut "en attente", gérer l'upload des photos (vérifier l'extension, sauvegarder dans `static/uploads/`, insérer le chemin dans la table `photos`). Route POST `/modifier/` et GET/POST `/supprimer/` : vérifier que c'est bien l'annonce de l'utilisateur connecté avant d'agir.

Fichiers : `app.py` · `static/uploads/`

Logique de modération (valider / rejeter une annonce)

Route accessible uniquement à un admin (vérifier `session["role"] == "admin"`). Afficher la liste des annonces en statut "en attente". Boutons pour passer le statut à "validée" ou "rejetée" dans la base de données. Seules les annonces validées apparaissent sur la page d'accueil publique.

Fichiers : `app.py` · `templates/admin.html`

P2

Personne 2 — niveau intermédiaire

Ce que l'utilisateur voit · Frontend + Jinja2

niveau moyen

Page d'accueil — liste des annonces

Créer le template Jinja2 de la page d'accueil. Afficher toutes les annonces validées sous forme de cartes (photo, titre, prix, wilaya, surface, prix/m²). Le prix au m² se calcule directement dans le template : `{{ (annonce.prix / annonce.surface) | int }}` DA/m². Ajouter un lien "voir le détail" sur chaque carte.

Fichiers : `templates/index.html` · `static/style.css`

Page détail d'une annonce

Template qui affiche toutes les infos d'une annonce : galerie de photos, description complète, prix, surface, prix/m², wilaya, type (vente/location), coordonnées du vendeur (seulement si connecté). Afficher aussi les autres annonces de ce vendeur (historique annonceur) pour la transparence des prix.

Fichiers : `templates/annonce.html`

Formulaire de publication d'annonce (pas-à-pas)

Créer le formulaire HTML guidé avec tous les champs : titre, type (vente/location), wilaya (liste déroulante des 58 wilayas), prix, surface, description, upload de photos (multiple). Ajouter des messages d'erreur Jinja2 si un champ est manquant (`{{ erreurs.titre }}`). Le formulaire envoie les données à la route `/publier` de P1.

Fichiers : `templates/publier.html` · `static/style.css`

Barre de recherche et filtres

Ajouter un formulaire GET en haut de la page d'accueil avec : champ texte (recherche par titre/description), liste déroulante wilaya, liste déroulante type (vente/location/tous), champ prix min et prix max. Ces filtres envoient les paramètres dans l'URL (`?wilaya=alger&type=vente`) — P1 se charge de la requête SQL côté Flask.

Fichiers : `templates/index.html`

Pages inscription et connexion

Deux templates simples avec des formulaires HTML : inscription (nom, email, téléphone, mot de passe, confirmation), connexion (email, mot de passe). Afficher les messages d'erreur Flask (identifiants incorrects, email déjà utilisé) via les flash messages Jinja2 : `{% for msg in get_flashed_messages() %}`.

Fichiers : `templates/inscription.html` · `templates/connexion.html`

Personne 3 — niveau débutant

P3

Ce qui relie tout · Tests, navigation, espace perso

moins expérimenté

Template de base (navbar + footer communs)

Créer un fichier `base.html` que tous les autres templates vont hériter avec `{% extends "base.html" %}`. Ce fichier contient : la navbar (logo, liens Accueil / Publier / Connexion / Inscription, afficher le nom de l'utilisateur si connecté via `{{ session.nom }}`), le footer, et l'inclusion du fichier CSS. C'est le squelette de toute l'appli.

Fichiers : `templates/base.html` · `static/style.css`

Espace personnel "Mes annonces"

Template qui affiche les annonces de l'utilisateur connecté (récupérées par P1 depuis la BDD). Pour chaque annonce : titre, statut (en attente / validée / rejetée) affiché avec une couleur différente, boutons Modifier et Supprimer. Page accessible uniquement si connecté (rediriger vers `/connexion` sinon).

Fichiers : `templates/mes_annonces.html`

Tests manuels de toutes les fonctionnalités

Tester chaque scénario dans le navigateur et noter les bugs : créer un compte → se connecter → publier une annonce avec photos → vérifier qu'elle apparaît en "en attente" → se connecter en admin → valider l'annonce → vérifier qu'elle apparaît sur l'accueil → rechercher avec les filtres → se déconnecter. Remonter chaque bug à P1 ou P2 avec une description précise.

Fichiers : `fichier_tests.txt`

Page d'erreur 404 personnalisée

Créer un template 404.html affiché quand une page n'existe pas. Enregistrer le gestionnaire d'erreur dans app.py avec `@app.errorhandler(404)`. C'est une tâche courte mais qui finit l'application proprement. Message simple : "Page introuvable" avec un lien retour à l'accueil.

Fichiers : *templates/404.html* · *app.py*

Déploiement sur le serveur Linux

Installer Python 3 et Flask sur le serveur (`pip install flask`), transférer les fichiers (via SCP ou clé USB), lancer l'appli avec `python app.py` en mode debug désactivé (`debug=False`), vérifier que l'appli est accessible depuis un autre PC du réseau via l'IP du serveur. Prendre des captures d'écran pour le rapport.

Fichiers : *serveur Linux* · *commandes bash*

À faire ensemble (synchro obligatoire)

- P1 crée le dépôt Git, P2 et P3 clonent
- P1 partage `init_db.py` dès le 1er jour
- P2 montre ses templates à P1 pour les relier
- P3 remonte les bugs à P1/P2 avec captures